Week 5 – Security Testing

Vulnerability Report

Introduction

Security testing was performed to identify vulnerabilities in the web application. The testing focused on detecting potential security weaknesses such as improper input validation, cross-site scripting attempts, authentication issues, and missing security configurations.

Both **automated security scanning** and **manual testing techniques** were used during this phase. Automated scanning was performed using OWASP ZAP, while manual testing was conducted on the SauceDemo login functionality.

Security Testing Tools

The following tool was used during the security testing phase:

OWASP ZAP

OWASP ZAP is an open-source security testing tool designed to find vulnerabilities in web applications. It performs automated scans to detect common web security issues such as missing security headers, cross-domain misconfigurations, and information disclosure.

The tool was configured to scan the SauceDemo website and generate vulnerability alerts.

Automated Security Testing

After scanning the SauceDemo application using OWASP ZAP, several vulnerabilities were detected.

Vulnerability ID: VULN-01

Description

Content Security Policy (CSP) Header Not Set. The application does not define a Content Security Policy header to control which resources the browser is allowed to load.

Severity

Medium

Steps to Reproduce

1. Open OWASP ZAP.
2. Enter the target URL of the SauceDemo website.
3. Start the automated scan.
4. Navigate to the **Alerts** section.
5. Locate the alert titled **Content Security Policy Header Not Set**.

Evidence

Suggested Fix

Configure the server to include a Content-Security-Policy HTTP header to restrict resource loading.

Vulnerability ID: VULN-02

Description

Cross-Domain Misconfiguration. Improper cross-domain configuration may allow external domains to access application resources.

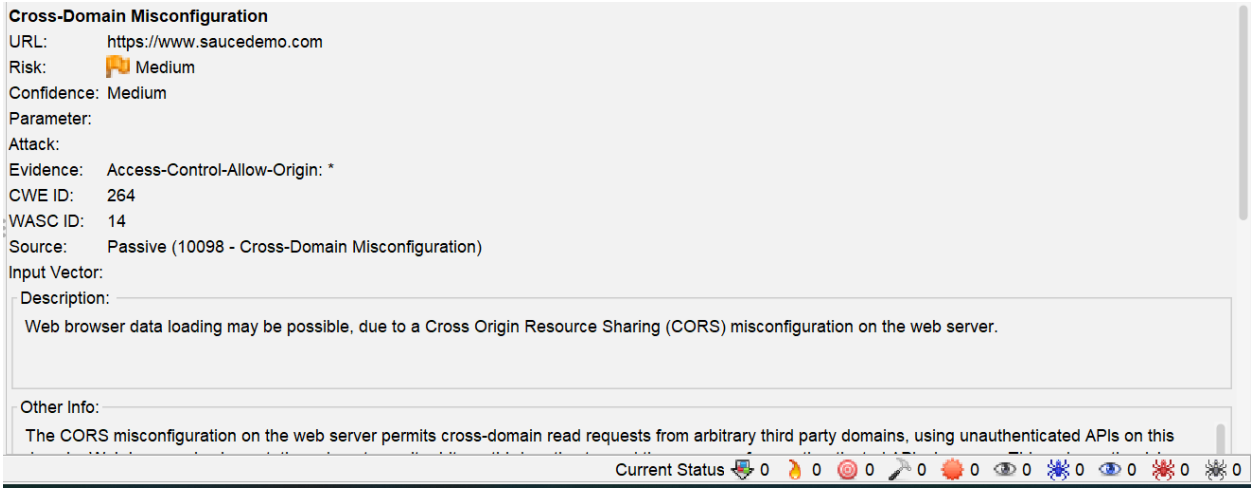
Severity

Medium

Steps to Reproduce

Run the OWASP ZAP scan and review the alert **Cross Domain Misconfiguration**.

Evidence



Suggested Fix

Restrict cross-origin requests and configure a proper **CORS policy**.

Vulnerability ID: VULN-03

Description

Missing Anti-Clickjacking Header. The application does not include the X-Frame-Options header which protects against clickjacking attacks.

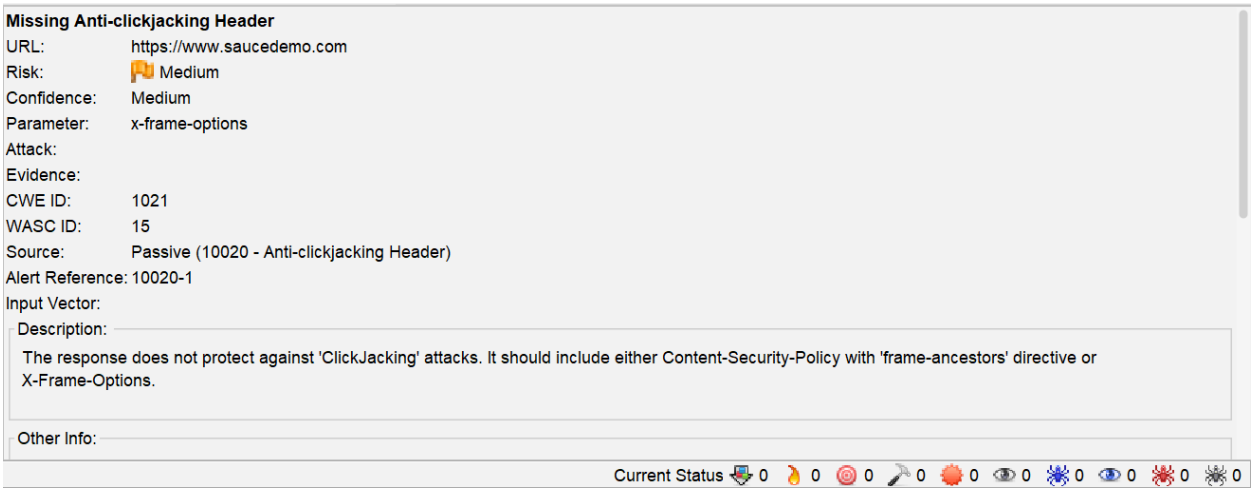
Severity

Medium

Steps to Reproduce

Run the automated scan in OWASP ZAP and observe the alert titled **Missing Anti-Clickjacking Header**.

Evidence



Missing Anti-clickjacking Header

URL: https://www.saucedemo.com
Risk: Medium
Confidence: Medium
Parameter: x-frame-options
Attack:
Evidence:
CWE ID: 1021
WASC ID: 15
Source: Passive (10020 - Anti-clickjacking Header)
Alert Reference: 10020-1
Input Vector:

Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Other Info:

Current Status 0 0 0 0 0 0 0 0 0 0

Suggested Fix

Add the following header in the server configuration:

X-Frame-Options: SAMEORIGIN

Vulnerability ID: VULN-04

Description

The scan detected that external resources such as JavaScript or CSS files are loaded without using a **Sub Resource Integrity (SRI) attribute**.

Subresource Integrity allows browsers to verify that files fetched from external sources have not been tampered with. Without this attribute, if a third-party resource is compromised, malicious code could be executed in the user's browser.

Severity

Medium

Steps to Reproduce

Run the automated scan in OWASP ZAP and observe the alert **“Sub Resource Integrity Attribute Missing.”**

Evidence

Sub Resource Integrity Attribute Missing

URL: https://www.saucedemo.com
Risk: Medium
Confidence: High
Parameter:
Attack:
Evidence: <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=DM+Sans:wght@400;500">
CWE ID: 345
WASC ID: 15
Source: Passive (90003 - Sub Resource Integrity Attribute Missing)
Input Vector:

Description:

The integrity attribute is missing on a script or link tag served by an external server. The integrity tag prevents an attacker who have gained access to this server from injecting a malicious content.

Other Info:

Suggested Fix

Developers should include the **integrity attribute** when loading external scripts or stylesheets.

Vulnerability ID: VULN-05

Description

The Strict-Transport-Security (HSTS) header is not set in the server response. Without this header, browsers may allow connections over HTTP, which can expose users to man-in-the-middle attacks.

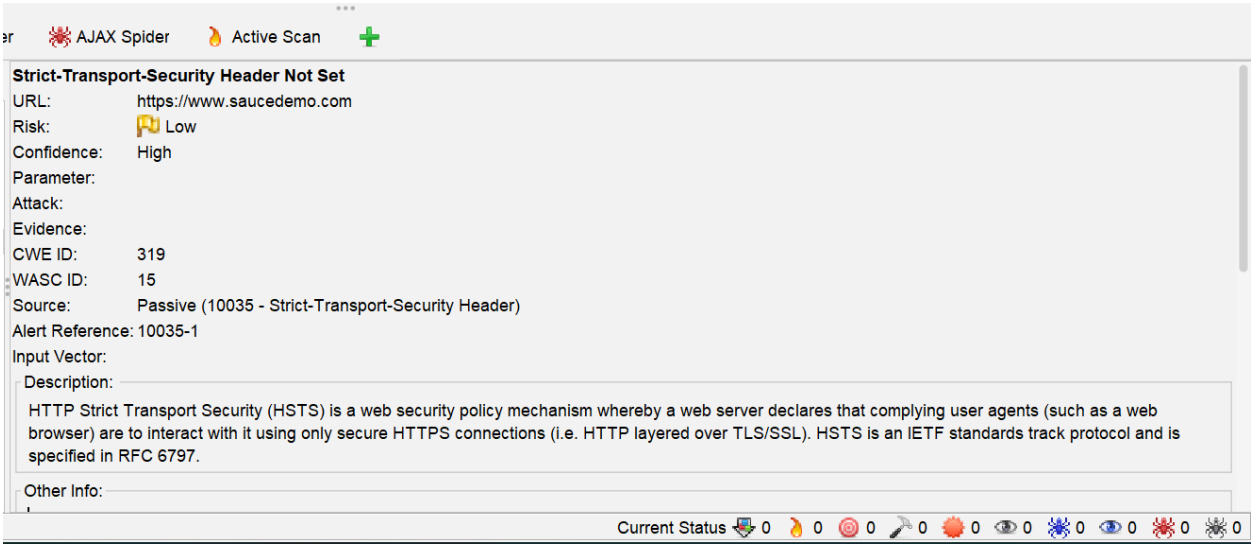
Severity

Low

Steps to Reproduce

1. Perform a scan using OWASP ZAP.
2. Review the alert titled **Strict-Transport-Security Header Not Set**.

Evidence



Suggested Fix

Ensure that your web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

Vulnerability ID: VULN-06

Description

X-Content-Type-Options Header Missing. Without this header, browsers may attempt to interpret files as a different content type.

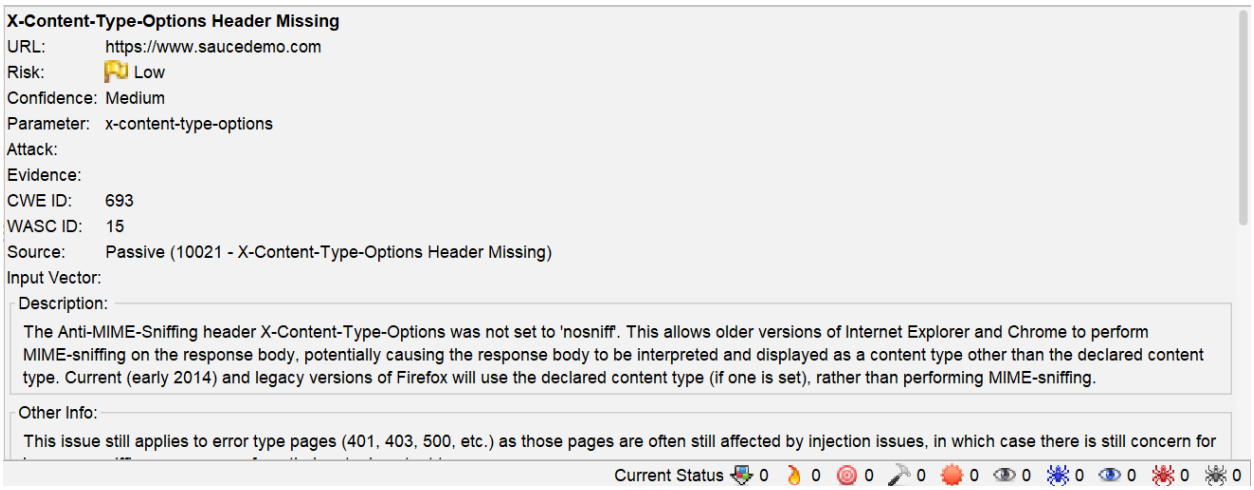
Severity

low

Steps to Reproduce

Run the automated scan and locate the alert **X-Content-Type-Options Header Missing**.

Evidence



X-Content-Type-Options Header Missing

URL: https://www.saucedemo.com
Risk: Low
Confidence: Medium
Parameter: x-content-type-options
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10021 - X-Content-Type-Options Header Missing)
Input Vector:

Description:

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Other Info:

This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for

Current Status 0 0 0 0 0 0 0 0 0

Suggested Fix

X-Content-Type-Options: nosniff

Vulnerability ID: VULN-07

Description

Information Disclosure – Suspicious Comments. The source code contains comments that may reveal internal information about the system.

Severity

Informational

Steps to Reproduce

1. Perform a scan using OWASP ZAP.
2. Review the alert titled **Information Disclosure - Suspicious Comments**.

Evidence

Suggested Fix

Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

Manual Security Testing

In addition to automated scanning, manual security testing was conducted to verify the application's behavior against common attack techniques.

Vulnerability ID: MAN-01

Description

Weak Input Validation Testing.

The login form was tested with invalid and malicious inputs to evaluate whether the system properly validates user input.

Severity

Low

Steps to Reproduce

Username

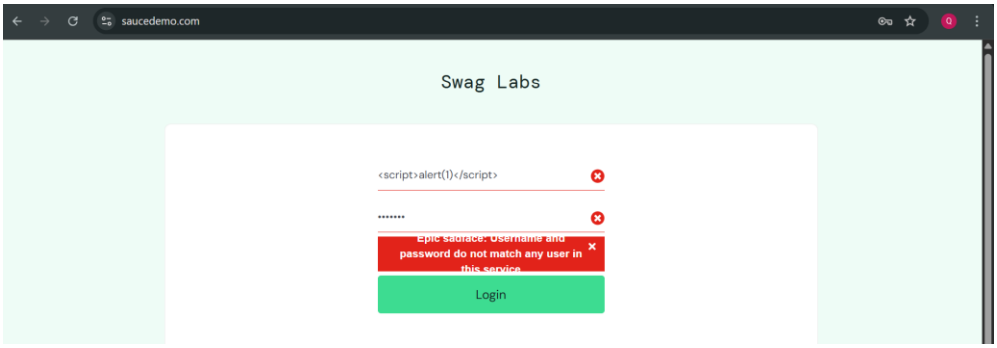
<script>alert(1)</script>

Password

test123

Submit the login form.

Evidence



Suggested Fix

Implement strict input validation and sanitization on user inputs.

Vulnerability ID: MAN-02

Description

Cross-Site Scripting (XSS) Testing.

The login input field was tested with a basic XSS payload.

Severity

Low

Steps to Reproduce

Username

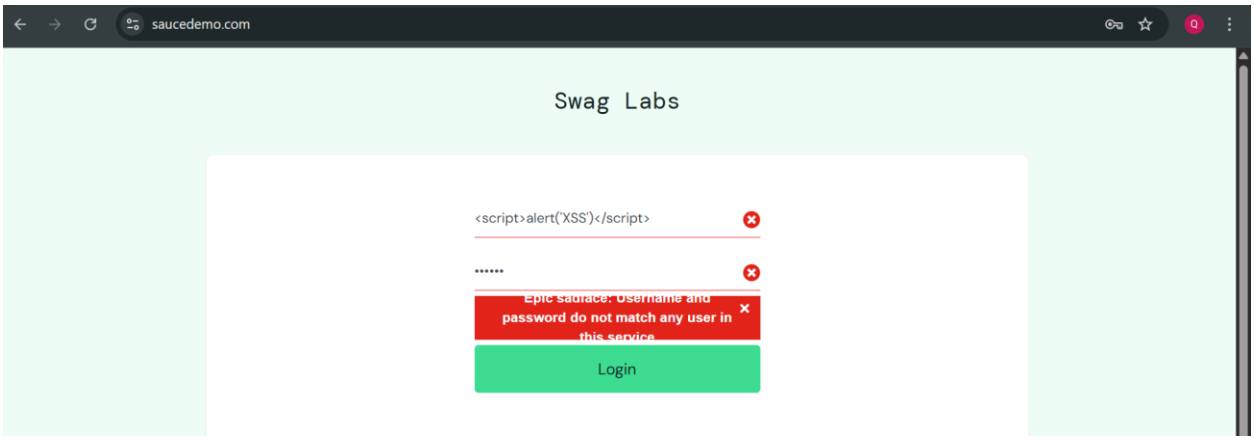
<script>alert('XSS')</script>

Password

123456

Click Login.

Evidence



Suggested Fix

Continue implementing output encoding and input validation to prevent XSS attacks.

Vulnerability ID: MAN-03

Description

Session Timeout Behavior.

The application session remained active after a long period of inactivity.

Severity

Low

Steps to Reproduce

1. Login using valid credentials.

Username

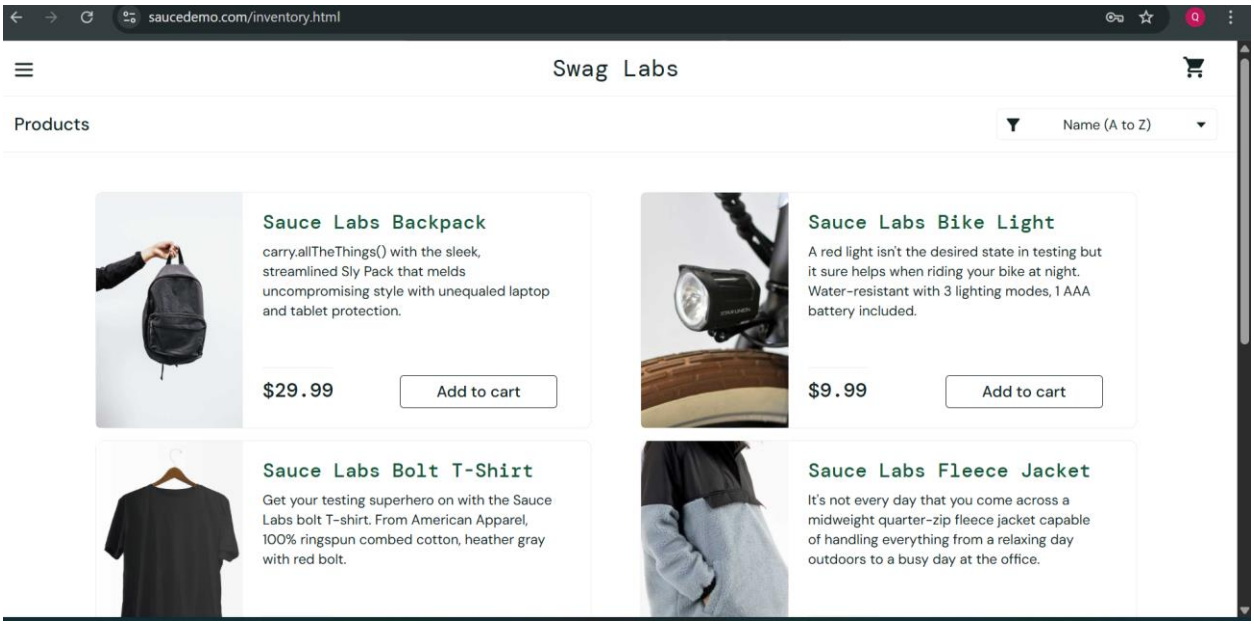
standard_user

Password

secret_sauce

2. Leave the session idle for 10–15 minutes.
3. Attempt to access another page.

Evidence



Suggested Fix

Implement automatic session timeout after a defined inactivity period.

Conclusion

Security testing of the application was conducted using both automated and manual techniques. The automated scan performed with OWASP ZAP identified several vulnerabilities, mainly related to missing security headers and configuration weaknesses in the SauceDemo.

Manual testing was also performed to evaluate input validation, cross-site scripting protection, authentication behavior, and session management. The application handled malicious input attempts correctly, and no successful XSS attacks were observed during testing. However, improvements such as implementing stronger security headers and enforcing session timeout policies could further enhance the overall security of the system.